Supplementary Material

Efficient Vision-Language Models: A Quantitative Survey of 1,947 arXiv Papers with Evidence Grading and Consumer-GPU Measurement

Every number below is produced by the released scripts; file names refer to the repository https://github.com/fofifafo/efficient-vlm-survey-2026 (archived at https://doi.org/10.5281/zenodo.23636019).

S1. Corpus retrieval

22 queries to the arXiv API, submissions from 2024-01-01 to 2026-08-31, de-duplicated by arXiv identifier (18,142 records). abs: restricts a term to the abstract.

```
Q1. abs:"vision-language model"
Q2. abs:"multimodal large language model"
Q3. abs:"visual instruction tuning"
Q4. abs:"efficient MLLM"
Q5. abs:"efficient vision-language"
Q6. abs:"token pruning" AND abs:multimodal
Q7. abs:"visual token" AND abs:compression
Q8. abs:"on-device" AND abs:"vision language"
Q9. abs:"mobile" AND abs:"vision-language model"
Q10. abs:"lightweight" AND abs:MLLM
Q11. abs:"KV cache" AND abs:multimodal
Q12. abs:quantization AND abs:MLLM
Q13. abs:"any resolution" AND abs:multimodal
Q14. abs:"high-resolution" AND abs:MLLM
Q15. abs:"document understanding" AND abs:multimodal
Q16. abs:"video" AND abs:MLLM
Q17. abs:"visual grounding" AND abs:"large language model"
Q18. abs:"multimodal hallucination"
Q19. abs:"multimodal" AND abs:"chain-of-thought"
Q20. abs:"retrieval-augmented" AND abs:multimodal
Q21. abs:"vision-language-action"
Q22. abs:"open-vocabulary" AND abs:segmentation
```

VLM papers (15,394): title or abstract matches any of the following patterns (case-insensitive).

```
mllm: multimodal large language model|MLLM|multi-modal LLM|multimodal LLM
vision-language-model: vision[- ]language model|VLM\b
visual-instruction-tuning: visual instruction tuning
```

Efficiency-oriented papers (1,947): VLM papers that carry at least one of the 12 efficiency keyword tags below.

```
efficient-vlm: efficient (MLLM|VLM|multimodal|vision[- ]language)
knowledge-distillation: knowledge distillation|distill
kv-cache: KV[- ]cache|key[- ]value cache
lightweight-vlm: lightweight (MLLM|VLM|vision[- ]language|multimodal)
low-power: low[- ]power|energy[- ]efficient|power efficient
mobile-vlm: mobile (VLM|vision[- ]language|MLLM)|MobileVLM
on-device: on[- ]device|edge deploy|edge device
peft-lora: \bLoRA\b|parameter[- ]efficient fine[- ]tuning|PEFT
quantization: quantiz
small-vlm: \b(small|tiny) (MLLM|VLM|vision[- ]language)|TinyLLaVA|small-scale VLM
token-merging: token merging|token merg|token compression|token packer|visual token compress
token-pruning: token pruning|token reduction|prune .*token
```

S2. Branch rules

Loose: the branch vocabulary appears in the title or abstract. Strict (4.1–4.4): the vocabulary appears in the title, or after a proposal phrase in the same sentence. Strict (4.5, 4.6): the separate patterns below. Title and abstract are joined as 'title. abstract'.

Proposal phrase:

```
(?:\bwe\b|this (?:paper|work|study))[^.]{0,40}?\b(?:propose|present|introduce|design|develop|build)[^.]{0,200}?
```

4.1 loose:

```
mixture[- ]of[- ]experts|\bMoE\b|early[- ]exit|layer skip|depth pruning|layer pruning|encoder[- ]free|lightweight
(?:vision|visual) encoder|small(?:er)? (?:vision|visual) encoder|state space model|\bMamba\b|linear
attention|efficient attention|\bQ[- ]?Former\b|resampler|perceiver
```

4.2 loose:

```
\bLoRA\b|\bQLoRA\b|\bDoRA\b|low[- ]rank adapt|parameter[- ]efficient|\bPEFT\b|\badapters?\b|prompt tuning|prompt
learning|data[- ]efficient|data selection|data curation
```

4.3 loose:

```
(?<!vector )(?<!vector-
)(?<!residual )(?<!product )quantiz(?![^.]{0,40}?(?:tokeniz|codebook|\btokens?\b|\baction))|\bPTQ\b|\bQAT\b|\bINT[48
]\b|W4A16|W8A8|W4A4|binariz|ternary|(?:knowledge distillation|teacher[-
 ]student)[^.]{0,150}(?:small|smaller|compact|lightweight|light-weight|tiny|compress|fewer
parameters|\d+(?:\.\d+)?\s?[MB]\b)|(?:small|smaller|compact|lightweight|light-weight|tiny|compress|fewer
parameters|\d+(?:\.\d+)?\s?[MB]\b)[^.]{0,150}(?:knowledge distillation|teacher[- ]student)|low[- ]rank
(?:compress|decompos|approximat|factoriz)|\bSVD\b|weight pruning|structured pruning|model
pruning|(?:weight|parameter|model|activation)[- ]sparsi|sparsif\w*
(?:the )?(?:weights|model|parameters|LLM|network)|(?:N:M|structured|unstructured|semi-structured) sparsity
```

4.4 loose:

```
(?:visual|vision|image|video) tokens? (?:prun|reduc|compress|merg|select)|token
(?:pruning|reduction|compression|merging|selection)|KV[- ]cache|key[- ]value cache|prefill|speculative
decoding|flash[- ]?attention|\bvLLM\b|paged attention|prefix cach|continuous batching
```

4.5 loose:

```
on[- ]device|edge device|edge deploy|Jetson|Raspberry|Snapdragon|smartphone|mobile
phone|\biPhone\b|\bOrin\b|\bNPU\b|\bDSP\b|\bFPGA\b|hardware accelerator|energy consumption|power
consumption|energy[- ]efficient|low[- ]power
```

4.5 strict:

```
(?:on|using|with) (?:a |an |the |single )?(?:NVIDIA )?(?:Jetson|Raspberry|Snapdragon|smartphones?|mobile
phones?|iPhone|Orin|Google Pixel|Pixel \d|NPU|FPGA)\b|(?:deployed|run|runs|running) on (?:a |an
|the )?(?:NVIDIA )?(?:Jetson|Raspberry|Snapdragon|smartphones?|mobile phones?|iPhone|Orin|Google Pixel|Pixel
\d|NPU|FPGA)\b
```

4.6 loose:

```
latency|inference time|wall[- ]clock|throughput|tokens per second|\bTPS\b|\bTTFT\b|\bFLOPs\b|\bMACs\b|memory
footprint|\bVRAM\b|power consumption|energy consumption
```

4.6 strict:

```
\d+(?:\.\d+)?\s*(?:ms|milliseconds|seconds? per|GB|GiB|MB|[GTM]?FLOPs|MACs|fps|FPS|tokens?/s|tokens per
second|TPS|W\b|watts?|J\b|joules?|mWh|mJ)\b|\d+(?:\.\d+)?\s?[x×]\s*(?:faster|speed-?up)|speed-?up of \d
```

Narrowing after validation. A validation against human labels (S5) showed that the compression rule and the deployment strict rule matched senses that the written screening criteria exclude: vector-quantization tokenizers, quantized visual or action tokens, token sparsification and distillation without a smaller student (compression); the word 'pixel' mistaken for the Google Pixel phone, and 'edge device' accepted without a named device (deployment). Both rules were narrowed to the criteria; no other rule changed. Before narrowing, the two rules were:

4.3 loose (before narrowing):

```
quantiz|\bPTQ\b|\bQAT\b|\bINT[48]\b|W4A16|W8A8|W4A4|binariz|ternary|knowledge distillation|teacher[- ]student|low[-
 ]rank (?:compress|decompos|approximat|factoriz)|\bSVD\b|weight pruning|structured pruning|model pruning|sparsif
```

4.5 strict (before narrowing):

```
(?:on|using|with) (?:a |an |the |single )?(?:NVIDIA )?(?:Jetson|Raspberry|Snapdragon|smartphone|mobile
phone|iPhone|Orin|Pixel|NPU|FPGA|edge (?:GPU|device))|deployed on (?:a |an
|the )?(?:Jetson|Raspberry|Snapdragon|smartphone|mobile|iPhone|Orin|edge|NPU|FPGA)|run(?:s|ning)? on (?:a |an
|the )?(?:Jetson|Raspberry|Snapdragon|smartphone|mobile|iPhone|Orin|edge device|NPU|FPGA)
```

S3. Sensitivity of the strict rule (all VLM papers)

Variant	Branch	2024 %	2026 %	×growth	Fisher p	Significant (α = 0.0042)
as in S2	4.1	1.19	1.14	0.96	8.3e-01	no
as in S2	4.2	5.53	3.26	0.59	7.6e-07	yes
as in S2	4.3	0.43	0.82	1.89	5.4e-02	no
as in S2	4.4	1.16	2.93	2.53	1.1e-07	yes
verbs propose/present/introduce only	4.1	1.16	1.08	0.93	7.4e-01	no
verbs propose/present/introduce only	4.2	5.31	3.09	0.58	7.5e-07	yes
verbs propose/present/introduce only	4.3	0.43	0.79	1.82	7.0e-02	no
verbs propose/present/introduce only	4.4	1.08	2.91	2.68	2.4e-08	yes
subject 'we' only, three verbs	4.1	1.12	1.08	0.96	8.3e-01	no
subject 'we' only, three verbs	4.2	5.17	3.04	0.59	1.5e-06	yes
subject 'we' only, three verbs	4.3	0.43	0.79	1.82	7.0e-02	no
subject 'we' only, three	4.4	1.08	2.91	2.68	2.4e-08	yes

verbs						
no title matches	4.1	0.87	0.92	1.06	9.0e-01	no
no title matches	4.2	4.12	2.64	0.64	2.7e-04	yes
no title matches	4.3	0.18	0.59	3.29	6.7e-03	no
no title matches	4.4	0.76	1.96	2.58	9.7e-06	yes

The proposal-phrase base rate (any proposal sentence in a VLM abstract) moved from 85.9% to 88.9% (×1.03); execution-time growth divided by it is ×2.45. Correcting over all 24 tests of both corpora ($\alpha = 0.0021$) leaves every significant result unchanged.

S4. Screening

Eligibility (topic fit to the branch criteria) was judged on title and abstract; selection for full reading then preferred papers with released code or a measured cost. A second rater judged eligibility for 50 candidates (10 per branch) without access to the first rater's decisions: Cohen's $\kappa = 0.70$ (90% agreement). Code availability shaped selection (39.8% of candidates with a code link kept, 4.2% without).

Eligibility criteria per branch (English rendering of main_survey/SCREENING_CRITERIA.md):

Branch	Eligible if the paper	Not eligible
4.1 Design	Proposes or compares mixture-of-experts, early exit, layer reduction, a lightweight or removed vision encoder, state-space models or linear attention, or connector design	Merely uses an existing MoE model
4.2 Training	Proposes or analyses a LoRA, PEFT, adapter, prompt-tuning or data-selection method itself	Uses LoRA as a tool to solve another task
4.3 Compression	Proposes or analyses a method that applies quantization, low-rank decomposition, distillation into a smaller student, or weight or structured pruning to a VLM	Vector-quantization tokenizers, data distillation, action quantization
4.4 Execution	Proposes or analyses visual-token compression, KV-cache, prefill or decoding acceleration, or serving techniques	Only mentions the number of tokens
4.5 Deployment	Runs or measures on a real device (Jetson, Snapdragon, smartphone, NPU, FPGA, etc.)	Aspirational statements such as 'deployable on edge devices'
4.6 Measurement	Concerns efficiency metrics, benchmarks or measurement methods themselves	(not used for selection; audit only)

First-round candidate lists. Each paper of the efficiency pool was scored for each branch by the weighted patterns below (a matching pattern adds its weight once) and assigned to its highest-scoring branch (ties to the earlier branch); 464 papers matched no pattern. Within each branch the papers were sorted by score (ties: most recent first), and the 60 highest-scoring papers and the 15 most cited of the rest (Semantic Scholar counts) were placed on cards: 59 (all) for design, and 75 of 445 for training, of 310 for compression, of 482 for execution and of 187 for deployment. Of the 1,947 efficiency papers, 461 were placed on a card in either round. These patterns only order the candidates; the trend analysis uses the rules of Section 3.2. Script: build_sheets.py; the check that it reproduces the first-round lists: screening_coverage.py.

First-round screening patterns and weights (case-insensitive regular expressions on title and abstract):

Branch	Pattern	Weight
4.1	mixture[-]of[-]experts|\bMoE\b	3
4.1	early exit|layer skip|depth pruning|layer pruning	3
4.1	encoder[-]free|lightweight (vision|visual) encoder|small(er)? (vision|visual) encoder	3
4.1	state space model|\bMamba\b|linear attention|efficient attention	2
4.1	\bQ[-]?Former\b|resampler|perceiver|projector design	2
4.2	\bLoRA\b|\bQLoRA\b|\bDoRA\b|low[-]rank adapt	3
4.2	parameter[-]efficient|\bPEFT\b	3
4.2	\badapter\b	2
4.2	prompt tuning|prompt learning|visual prompt	1
4.2	data[-]efficient|data selection|data curation|curriculum	1
4.3	quantiz|\bPTQ\b|\bQAT\b|\bINT[48]\b|W4A16|W8A8|binariz|ternary	3
4.3	knowledge distillation|self[-]distill|teacher[-]student	3
4.3	low[-]rank|matrix factoriz|\bSVD\b	2
4.3	sparsity|sparse (model|weight)|structured pruning|weight pruning	2
4.4	visual token|token pruning|token reduction|token compress|token merg|token packer	3
4.4	KV[-]cache|key[-]value cache|cache (compress|evict|reuse|prun)	3
4.4	token selection|token sampling|prune \w+ token	2
4.4	prefill	2
4.4	speculative decoding|flash[-]?attention|\bvLLM\b|paged attention|prefix cach|continuous batching	2
4.5	Jetson|Raspberry|Snapdragon|smartphone|mobile phone|\biPhone\b|Orin	3
4.5	\bNPU\b|\bDSP\b|\bFPGA\b|\bASIC\b|hardware accelerator	3

4.5	on[-]device|edge device|edge deploy	2
4.5	energy consumption|power consumption|energy[-]efficient	2
4.5	memory footprint|\bVRAM\b|memory usage	1

Second-round candidate lists. The design, training and deployment lists contain every paper of a rule-based candidate pool (44, 45 and 70 papers), each card with a model summary. The compression list was drawn differently: a title filter returned 190 papers, of which 67 meet the strict compression rule of Section 3.2; the model read their titles and the first 380 characters of their abstracts and chose 59 as papers that apply quantization, low-rank factorization, pruning or distillation to VLMs or vision-language-action models, and added two papers from other candidate lists, giving 61 cards. No written criteria governed this choice; the verdicts on the 61 cards followed the criteria above. Of the 131 papers not chosen, 35 appeared on other lists (3 kept) and 96 were never screened; they are listed in the release (main_survey/selection_190/h_unselected.csv), together with the script that reproduces the filter (selection_190_check.py). Of these 96, 41 lie outside the efficiency pool and 51 had been assigned to a branch in the first round but ranked below its cut-off (31 of them in compression, at ranks 98–304 of 310); 16 meet the strict compression rule of Section 3.2. A further deployment list re-presented 39 papers that had been skipped on the first deployment list, leaving out those whose model summary placed them outside the survey or in another branch; a pattern marked the 32 whose abstract states a run or measurement on a device. Summaries flagged 57 cards as recommended and ended on 58 with a suggestion to skip; 25 and 3 of these were kept, against 66 of the other 503 cards.

S5. Validation of the branch rules

Design. Precision: 300 abstracts from all VLM papers (6 branches × 2024/2026 × 25; per stratum 13 strict matches and 12 loose-only matches), labelled by the first author for (Q1) whether the paper concerns the branch in the intended sense and (Q2) whether it meets the strict criterion; loose precision is weighted by stratum size. Recall: 100 random efficiency-oriented abstracts labelled for all branches, scored against the final rules. The compression and deployment rules were narrowed after these samples had been examined, so their strict precision is taken from a fresh sample of 55 matches; their recall is computed on the original recall sample, which was seen when the rules were narrowed, and may be optimistic.

Table S5. Precision and recall of the branch rules (first author's labels).

Branch	Strict precision 2024	Strict precision 2026	Loose precision	Loose recall	Strict recall	Precision sample
4.1 Design	85% (11/13)	69% (9/13)	82%	43% (3/7)	25% (1/4)	original 300 (rule unchanged)
4.2 Training	85% (11/13)	69% (9/13)	80%	90% (27/30)	69% (9/13)	original 300 (rule unchanged)
4.3 Compression	33% (3/9)	54% (7/13)	57%	50% (7/14)	29% (2/7)	fresh 55 (after narrowing)
4.4 Execution	92% (12/13)	100% (13/13)	85%	81% (21/26)	62% (13/21)	original 300 (rule unchanged)
4.5 Deployment	50% (1/2)	80% (8/10)	53%	88% (7/8)	– (no positives)	fresh 55 (after narrowing)
4.6 Measurement	69% (9/13)	92% (12/13)	86%	52% (15/29)	19% (5/26)	original 300 (rule unchanged)

Second rater. The second author, who did not take part in designing the rules, judged a subsample from titles and abstracts under the same written criteria, without access to the first author's labels: 60 precision items (10 per branch), 20 recall papers and the 11 items of the fresh sample. Disagreements were not adjudicated; the labels above are the first author's.

Sample	Question	Items	Agreement	Cohen's κ
Precision (60)	Q1 concerns the branch	60	90%	0.71
Precision (60)	Q2 strict criterion	60	88%	0.77
Recall (20 papers × 6 branches)	branch addressed (loose)	120	93%	0.76
Recall (20 papers × 6 branches)	strict criterion	120	94%	0.71
Fresh sample (11)	Q1 concerns the branch	11	64%	0.31
Fresh sample (11)	Q2 strict criterion	11	82%	0.54

Branch	Items	Q1 agreement	Q2 agreement
4.1 Design	10	9/10	9/10
4.2 Training	10	7/10	7/10
4.3 Compression	10	10/10	8/10
4.4 Execution	10	10/10	10/10
4.5 Deployment	10	8/10	9/10
4.6 Measurement	10	10/10	10/10

Evidence grades. The second rater also judged, from the full texts, the two facts behind the grade (public code; a time or memory cost measured by the authors) for a stratified sample of 20 entries. Before the correction described in S6, agreement was 80% for code ($\kappa = 0.47$), 85% for the measured cost ($\kappa = 0.5$) and 70% for the grade ($\kappa = 0.47$). The three measured-cost

differences were costs our extraction had missed (SWIFT, MiniDrive, Eve) and were corrected; after the correction the measured-cost fact agrees for 20 of 20 entries. Of the four code differences, three are repositories released outside the paper (Rank-1 Expert Pool LoRA, SWIFT, VideoChat-Flash), which our rule counts and the rater's instructions, which asked for a link in the paper, did not; one (AutoSelect) is a repository that holds only a README, which the instructions grade as no code.

S6. Full-text extraction rubric

Applied to the full text of each of the 69 selected papers (71 branch entries), with the reference list removed.

Step 1 — candidate sentences

The text is split into sentences; every sentence of at most 600 characters that matches one of the 14 patterns below (case-insensitive) is kept as a candidate, at most eight per pattern and paper.

Pattern — Meaning — Regular expression

```
SPEED — speed-up or relative speed — \d+(?:\.\d+)?\s?[x×]\s*(?:faster|speed-?up|acceleration|reduction)|speed-?up
(?:of|by) \d|\d+(?:\.\d+)?\s?% (?:faster|reduction in (?:latency|time|FLOPs))
FLOP — FLOPs — \d+(?:\.\d+)?\s?[GTM]?FLOPs|FLOPs? (?:reduc|drop|decrease)\w* (?:by
|to )?\d+(?:\.\d+)?\s?%|\d+(?:\.\d+)?\s?% (?:of (?:the )?)?FLOPs
LAT — latency or time — \d+(?:\.\d+)?\s?(?:ms|milliseconds|seconds?|s)\b[^.]{0,60}(?:latency|per
(?:image|sample|token|frame)|inference|prefill|decod|TTFT)|(?:latency|inference time|prefill time|TTFT|time to first
token)[^.]{0,60}\d+(?:\.\d+)?\s?(?:ms|s|seconds?)
MEM — memory — \d+(?:\.\d+)?\s?(?:GB|GiB|MB)\b[^.]{0,40}(?:memory|KV|cache|VRAM)|(?:memory|KV
cache|VRAM)[^.]{0,60}\d+(?:\.\d+)?\s?(?:GB|GiB|MB|%)
THR — throughput — \d+(?:\.\d+)?\s?(?:tokens?/s|tokens per second|samples?/s|fps|FPS|QPS|images?/s)
TOK — token budget — \d+(?:\.\d+)?\s?% (?:of
(?:the )?)?(?:visual|vision|image|video)? ?tokens|(?:prun|reduc|compress|retain|keep|drop)\w*[^.]{0,40}\d+(?:\.\d+)?
\s?% of (?:the )?(?:visual |vision |image )?tokens|(?:from|to) \d{2,4} (?:visual |vision )?tokens|\d{2,4} tokens?
(?:per|/) (?:image|frame)
HW — hardware —
\b(?:NVIDIA )?(?:A100|A800|H100|H800|V100|A6000|A40|L40S?|RTX ?\d{4}|3090|4090|T4|Jetson[\w ]{0,15}|Orin|Snapdragon[
\w ]{0,10}|Apple M\d|iPhone[\w ]{0,6}|TPU ?v\d|Ascend[\w ]{0,6}|CPU)\b
ACC — accuracy retention —
(?:accuracy|performance|score)[^.]{0,60}(?:drop|loss|degrad|decreas|retain|maintain|preserv|within)[^.]{0,40}\d+(?:\
.\d+)?\s?%|\d+(?:\.\d+)?\s?% (?:of
(?:the )?)?(?:original|baseline|full)[^.]{0,20}(?:performance|accuracy)|(?:retain|maintain|preserv|keep)\w*
\d+(?:\.\d+)?\s?%
FREE — training-free claim — training-?free|plug-?and-?play|without (?:any )?(?:re-?)?training|no (?:additional
|extra )?training|fine-?tun\w+ (?:is )?(?:required|needed)
BASE — base model — \b(?:LLaVA(?:-(?:1\.5|NeXT|OneVision|Video))?|Qwen2?(?:\.5)?-VL|InternVL[\d.]*|MiniCPM-V|Phi-3-
vision|BLIP-?2|Video-LLaVA|Mini-?Gemini|mPLUG-Owl|Idefics\d?|PaliGemma|Gemma ?3)\b
CMP — compared method — \b(?:FastV|SparseVLM|PruMerge|LLaVA-
PruMerge|ToMe|TokenPacker|PyramidDrop|VisionZip|DivPrune|FitPrune|VTW|CDPruner|TopV|HiRED|FasterVLM|Dynamic-
LLaVA|LOOK-M|VL-Cache|MustDrop|SparseVILA|FastVLM|VoCo-LLaMA|LLaVA-Mini|TRIM)\b
BIT — bit width — \b(?:W\d+A\d+|INT[248]|\d-bit|\d bit|ternary|binar\w+|FP8|NF4)\b
PARAM — parameter share — \d+(?:\.\d+)?\s?%\s?(?:of (?:the )?)?(?:trainable |total )?param\w*|\d+(?:\.\d+)?\s?[MB]
(?:trainable )?param\w*
BENCH — benchmark — \b(?:MME|MMBench|MMMU|GQA|VQAv2|TextVQA|POPE|ScienceQA|SEED-
?Bench|VizWiz|OCRBench|DocVQA|ChartQA|Video-
?MME|MVBench|MLVU|LongVideoBench|EgoSchema|MSVD|MSRVTT|ActivityNet|RefCOCO\w*|AI2D|MathVista|RealWorldQA)\b
```

Step 2 — reading and recording

A language model (Claude, Anthropic; interactive session, September–October 2026) read the candidate sentences in the context of the full text and filled one row per entry with the fields below. Instruction (rules):

1. Record a time, memory, throughput or energy quantity only when the authors measured it on their own system; quantities quoted from other papers and theoretical estimates are not recorded. FLOPs are recorded in their own field but do not count as measured.

2. Copy each quantity in the paper's own units and wording, together with the sentence (or table caption) that supports it.

3. Leave a field empty ('—') when the paper reports no such quantity; do not infer.

4. Name the hardware only as the paper names it.

Field — Definition
name — method name used by the authors
sub — sub-branch (Tables 3–7, column 3)
reduction — what is reduced and by how much: token budget (4.4), bit width (4.3), active/total parameters (4.1), trainable-parameter share (4.2), device budget (4.5)
flops — FLOPs reported by the authors (computed, not measured; does not count towards the grade)
speed_latency — time or speed-up measured by the authors on their own system
memory — memory measured by the authors on their own system

hardware — the devices named for those measurements; '—(not stated)' if none
accuracy_retained — accuracy relative to the unmodified model, as reported
compared_with — baselines in the efficiency comparison
code — public code link given in the paper

Step 3 — grading and checks

Grade A = public code and a measured time or memory cost; B = such a measurement without public code; C = no measured cost (FLOPs alone do not count).

Checks: a random sample of 30 extracted quantities compared with the papers (30/30 matched; main_survey/extraction_check/); every code link opened automatically and its repository listed (main_survey/check_code_links.py).

Extraction check: 30 of 30 randomly sampled quantities matched the papers, 24 at once and 6 after the model pointed to the table or figure that holds them and the first author confirmed them there. Code links: 57 of 57 resolve; 55 repositories contain source code, Kimi-VL releases model code with its weights on Hugging Face, and AutoSelect's repository holds only a README (graded B).

Grade correction. The second rater's check (S5) found measured costs in three entries graded C. All remaining grade-C entries were then re-read for time or memory quantities in the whole paper, including appendices and table cells: the original candidate sentences stopped at the reference list and missed units printed only in table cells. Eleven papers (12 entries) were regraded; the rule itself did not change (training time counts as a time cost; FLOPs, device capacity and quoted figures do not). Kimi-VL, which states only that training throughput is about 60% higher than a dense 7B model without a number or setting, was kept at C. Grades changed from 36 A, 11 B and 24 C to 46 A, 13 B and 12 C.

Branch | arXiv	Added time / memory	Supporting text
4.3|2408.05517	training speed LoRA 0.95, full 1.43, LISA 2.66 samples/s | training memory LoRA 32.35, full 73.53, LISA 31.11 GiB (Qwen-7B-chat)	4.1 Lightweight Tuning Benchmark: 'conducted training on a single A100-80G GPU, comparing memory usage and loss'; Table 4 (Memory (GiB), Speed (samples/s))
4.1|2409.07267	GPU memory 1.03 GB (MiniDrive224)	4.3 Computational Analysis / Table 2 'GPU memory usage': 'MiniDrive224 (ours) 83M 5.9B 1.03'
4.5|2501.04322	training 15 GPU-days	Table 5 GPU-days column; 'offers significant advantages in training efficiency, requiring only 15 GPU-days'
4.4|2409.10197	recipe search ≈5 min; throughput +22.2% at 50% pruning	Abstract: 'the pruning recipe can be obtained in about 5 minutes'; appendix: 'at a 50% pruning ratio … throughput increasing by 22.2%'; Table 7 throughput in samples per second
4.2|2405.18541	training 50 min vs 2 h for CoOp (16-shot ImageNet)	Table 1: 'Training time on 16-shots ImageNet task. Experiments were conducted on a single A100 80Gb' — CoOp (16) 2h, CLIP-LoRA 50 min
4.1|2408.15881	mimic distillation 960 vs 1,536 GPU hours (D2D+D2S vs D2S)	'D2D+D2S is more computationally efficient, consuming only 62.5% of the GPU hours required by D2S' (Table 7: 1536 vs 960); appendix: 'train on 16 NVIDIA A100 GPUs … approximately 960 GPU hours'
4.3|2408.15881	mimic distillation 960 vs 1,536 GPU hours (D2D+D2S vs D2S)	same as 4.1 entry
4.1|2412.10302	training 7/10/14 days on 16/33/42 nodes of 8 A100 GPUs	'The training of DeepSeek-VL2 was completed in 7/10/14 days using a cluster of 16/33/42 nodes, with each node equipped with 8 NVIDIA A100 GPUs.'
4.2|2508.00230	training per epoch 16.5 min (ViT-B/32), 137.5 min (ViT-L/14), 220.1 min (ViT-H/14); 247.6 min (LLaMA3-8B)	Appendix E 'Training times and VRAM usage', Table 5: KRAdapter 16.5 / 137.5 / 220.1 / 247.6 / 226.3 mins
4.2|2406.11909	training 8.2 h (LLaMA-3 8B, commonsense) | GPU memory 31 GB	Table 4: 'Time for the training time on A100 GPU, and Mem for the GPU Memory usage' — MoSLoRA 28.4M 8.2h 31G
4.4|2506.17218	training ≈3.5 h (stage 1) and ≈7.2 h (stage 2) vs 5.5 h for text-only CoT SFT	Appendix: 'Both training stages of Mirage are conducted on a single NVIDIA H100 GPU … Stage 1 completes in approximately 3.5 hours, while Stage 2 takes around 7.2 hours.'
4.2|2605.06175	action-chunk latency at batch 1, mean of 120 runs (values only in a figure of the paper)	Appendix D.4: 'measuring the wall-clock latency required to generate one action chunk … batch size 1 … NVIDIA RTX 5080 GPU … average latency over 120 runs' (Figure 5)

S7. Measurement

Method	Repository	Commit	Note

CDPruner	https://github.com/Theia-4869/CDPruner	9541616c40	latest commit on or before the clone date 2026-10-01 (GitHub API commits?until=2026-10-01); local copy has no .git
VisPruner	https://github.com/Theia-4869/VisPruner	aefa01adc7	same
SparseVLM	https://github.com/Gumpest/SparseVLMs	a9e7142722	same
DART	https://github.com/ZichenWen1/DART	0c88867319	same
FastV	https://github.com/pkunlp-icler/FastV	d1659729b5	same

Run-to-run variation: interquartile range of the five run medians below 2% of the median in 24 of 28 settings, at most 6.3%; CDPruner as released up to 58%.

Published accuracy used in Table 10 (LLaVA-1.5-7B, same token budget):

Method	**Tokens**	**GQA**	**POPE**	**TextVQA**	**MME-P**	**MME total**	**Source**
base	576	62.0	85.9	58.2	1510.7		VisPruner (arXiv 2412.01818) Table 1 upper bound; LLaVA-1.5-7B
vispruner	128	58.2	84.6	57.0	1461.4		arXiv 2412.01818 Table 1 (LLaVA-1.5-7B; Retain 128)
vispruner	64	55.4	80.4	55.8	1369.9		arXiv 2412.01818 Table 1 (LLaVA-1.5-7B; Retain 64)
cdpruner_fix	128	59.9	87.7	56.2	1431.4		arXiv 2506.10967 Table 1 (LLaVA-1.5-7B; Retain 128)
cdpruner_fix	64	58.6	87.5	55.3	1415.1		arXiv 2506.10967 Table 1 (LLaVA-1.5-7B; Retain 64)
sparsevlm	192	59.5	85.3	57.8		1787	arXiv 2410.04417 Table 1 (LLaVA-1.5-7B; Retain 192); MME reported as perception+cognition total
sparsevlm	64	53.8	77.5	53.4		1589	arXiv 2410.04417 Table 1 (LLaVA-1.5-7B; Retain 64); MME total
dart	128	58.7	80.1	56.4		1840	arXiv 2502.11494 Table 1 (LLaVA-1.5-7B; Retain 128); MME total
dart	64	55.9	73.9	54.4		1765	arXiv 2502.11494 Table 1 (LLaVA-1.5-7B; Retain 64); MME total
fastv	288						arXiv 2403.06764: LLaVA-1.5-7B results only for Nocaps/Flickr30k/A-OKVQA/MMMU; MME only for LLaVA-1.5-13B
fastv	64						arXiv 2403.06764: no 64-token setting

Jetson Orin Nano: 4-bit NF4 overhead on a 4096×11008 layer 4.1–4.7 ms at 16–1,024 tokens; token sweep over 6 visual-token counts (32–576), 100 questions × 3 runs, run-to-run spread within 0.5%.

Method	**Visual tokens**	**TTFT median (ms)**	**TTFT p90 (ms)**	**Energy (J/answer)**	**Mean power (W)**
base4	576	1964.9	2140.9	46.88	19.84
vispruner4	384	1633.9	1643.0	38.42	19.43
vispruner4	256	1402.8	1414.7	32.78	18.81
vispruner4	128	1198.7	1209.2	27.43	17.8
vispruner4	64	972.0	1162.1	23.8	17.33
vispruner4	32	950.4	965.5	21.73	16.94

Table S6. Time to first token split into GPU time (union of CUDA kernel and copy intervals in a profiled run) and host time (the remainder); median over 95 GQA questions (100, first five discarded). The RTX 5080 unmodified-model row was re-run after closing desktop applications that held 1.2–2.3 GB of GPU memory (first run 133.6 ms, kept in the repository). TTFT, GPU and host time are medians taken separately over questions, so GPU + host need not equal TTFT; the host shares quoted in Section 5.3 are medians of the per-question host/TTFT ratios. The GPU-time speed-

ups quoted in Section 5.5 (1.38–3.11× on the RTX 5080, 1.43–3.42× on the RTX 5070 Ti) are the GPU time of the unmodified model divided by that of each setting.

Method	Tokens	5080 TTFT	5080 GPU	5080 host	5070 Ti TTFT	5070 Ti GPU	5070 Ti host
LLaVA-1.5-7B	576	113.9	96.6	10.1	125.2	115.0	9.1
VisPruner	128	66.2	39.6	21.7	62.9	43.6	18.7
VisPruner	64	58.4	31.1	22.9	54.1	33.6	19.2
CDPruner (fix)	128	80.7	42.5	33.7	75.3	45.1	28.6
CDPruner (fix)	64	60.9	32.1	25.4	62.5	42.1	21.6
SparseVLM	192	84.2	52.9	30.9	84.4	59.1	24.7
SparseVLM	64	67.7	37.9	30.1	70.9	41.4	28.4
DART	128	75.0	49.5	25.4	75.1	53.6	21.6
DART	64	61.7	40.1	21.6	64.4	44.9	19.6
FastV (KV cache, layer 3)	288	86.8	70.2	16.9	95.8	80.6	14.7
FastV (KV cache, layer 3)	64	61.6	38.8	17.2	61.1	43.2	16.9

CDPruner memory diagnosis (45 GQA questions after five warm-up questions; memory after 20 questions). torch.cuda.Stream() hands out streams from a pool of 32 per device and the caching allocator keeps freed blocks per stream, so the released code's two side streams per call accumulate cached memory; no allocation retries or out-of-memory errors occurred under WSL2. Reported to the authors as GitHub issue #24.

GPU	CDPruner	Allocated (GB)	Reserved (GB)	Free (GB)	TTFT median (ms)	TTFT p90 (ms)
RTX 5080	released, 128	13.75	15.29	0.0	103	891
RTX 5080	released, 64	13.75	15.35	0.0	88	831
RTX 5080	default stream, 128	13.46	13.81	0.24	78	125
RTX 5070 Ti	released, 128	13.75	15.28	0.0	284	390
RTX 5070 Ti	released, 64	13.75	15.35	0.0	319	416
RTX 5070 Ti	default stream, 128	13.46	13.81	0.64	74	92

S8. Use of language models

A language model (Claude, Anthropic; Opus 5 family, interactive sessions) assisted the full-text extraction under the rubric in S6 and pointed to the location of 6 of the 30 quantities in the extraction check; every extracted quantity is stored with its supporting sentence. The model also provided reading aids, and all verdicts and labels were made by people: on the screening desk, a translation of each abstract when available and, on the 259 second-round cards, a one-line summary (57 cards flagged 'recommended' and 58 ending with a suggestion to skip); in the review of the five rater disagreements, a summary of each abstract; and, for the first author's labels of the 55 abstracts that validate the revised rules (S5), a Korean translation of each title and abstract without comment or summary, with the labels referring to the original text. The model also drew the second-round compression list from the titles and abstract openings of a title-filtered set (S4). The second raters worked without model assistance. In the grade re-check (S6) the model listed candidate sentences with time or memory units from the full texts and proposed the regradings with their supporting sentences; the first author reviewed and approved them.

S9. Robustness checks

Table S7. Execution-time branch (4.4) under the base vocabulary and a broader one that adds verb-before-noun phrasings, further verbs (drop, discard, remove, withdraw, condense, sparsify, select, evict), 'fewer visual tokens' and 'visual token budget'. Shares are of all VLM papers or of efficiency papers per year; p from Fisher's exact test 2024 vs 2026, significant at the Bonferroni level 0.05/12. Recall: first rater's labels on the 100-paper recall sample (existing labels only).

Corpus	Vocabulary	Definition	Papers 2024/2025/2026	Share 2024→2026 (%)	Growth	p	Significant	Recall 2024	Recall 2026
efficiency	base	loose	52/177/250	19.4→26.57	×1.37	1.7e-02	no	9/12	12/14
efficiency	base	strict	30/119/165	11.19→17.53	×1.57	1.4e-02	no	5/10	8/11
efficiency	broad	loose	60/184/257	22.39→27.31	×1.22	1.2e-01	no	11/12	12/14
efficiency	broad	strict	40/121/172	14.93→18.28	×1.22	2.4e-01	no	8/10	8/11
VLM	base	loose	61/210/284	2.2→4.57	×2.07	2.5e-08	yes	9/12	12/14
VLM	base	strict	32/131/182	1.16→2.93	×2.53	1.1e-07	yes	5/10	8/11
VLM	broad	loose	89/248/321	3.22→5.16	×1.6	3.0e-05	yes	11/12	12/14
VLM	broad	strict	49/146/195	1.77→3.14	×1.77	1.8e-04	yes	8/10	8/11

Influential methods: PyramidDrop — base loose=False strict=False, broad loose=True strict=False; VTW — base loose=False strict=False, broad loose=True strict=True; VoCo-LLaMA — base loose=False strict=False, broad loose=True strict=True; TokenPacker — base loose=False strict=False, broad loose=True strict=True; FasterVLM/VisPruner — base loose=True

strict=True, broad loose=True strict=True; LLaVA-PruMerge — base loose=True strict=True, broad loose=True strict=True. FasterVLM is the earlier title of VisPruner (arXiv 2412.01818). Adding 'large multimodal model(s)' and 'LMM' to the VLM definition would add 125 of the 18,142 records.

Abstract audit (Section 4.6). Applied to the abstracts of the 1,947-paper efficiency pool, case-insensitive. Metric terms, one pattern per category:

```
latency: latency|inference time|inference speed|wall[- ]clock|response time|time[- ]to[- ]first[- ]token|\bTTFT\b
device: \b(?:Jetson|Raspberry Pi|Snapdragon|Dimensity|Orin|iPhone|Pixel
\d|Galaxy|A100|H100|H800|A800|V100|A6000|RTX ?\d{4}|NPU|FPGA|smartphone|mobile phone)\b
params: \d+(?:\.\d+)?\s?(?:[BM]|billion|million)\s+(?:trainable\s+)?(?:parameters|params)|(?:number
of|fewer|reduc\w*|trainable) parameters
flops: \b(?:[GTM]?FLOPs?|MACs)\b|floating[- ]point operations
throughput: throughput|tokens? per second|tokens?/s\b|\bTPS\b|frames per second|\bFPS\b
memory: memory (?:footprint|usage|consumption|cost|overhead|requirement|saving|reduction)|\bVRAM\b|GPU memory|peak
memory|KV[- ]cache (?:size|memory|footprint)
energy: energy (?:consumption|efficien|cost)|power consumption|\bwatts?\b|\bjoules?\b|battery
```

Broader rule (any speed-up factor, percentage reduction or unit-bearing number):

```
\d+(?:\.\d+)?\s?(?:×|x|times)\s*(?:faster|speed|acceleration|higher
throughput|fewer|smaller|less|reduction|compression|lower)|(?:speed-?ups?|acceleration|faster)\s+(?:of|by|up
to)?\s*(?:up to\s*)?\d+(?:\.\d+)?\s?(?:×|x|%|times)|\d+(?:\.\d+)?\s?(?:×|x)\s+(?:speed-
?up|inference)|\d+(?:\.\d+)?\s?%\s+(?:fewer|less|lower|reduction|of the (?:visual |vision
|image )?tokens|tokens|FLOPs|memory|latency|parameters|computation|compute|faster|speed)|(?:reduc\w+|cut\w*|sav\w+|p
run\w+|compress\w*|remov\w+|decreas\w+|lower\w*)\b[^.]{0,60}?\bby\s+(?:up
to\s+|over\s+|about\s+|nearly\s+)?\d+(?:\.\d+)?\s?(?:%|×|x)|\d+(?:\.\d+)?\s?(?:ms|milliseconds?|seconds?|GB|GiB|MB|t
okens?/s|FPS|fps|W|J|mJ)\b|\b\d+(?:\.\d+)?-
?bit\b|\d+(?:\.\d+)?\s?(?:[BM]|billion|million)\s+(?:trainable\s+)?(?:parameters|params)|\d+(?:\.\d+)?\s?(?:G|T|M)?F
LOPs|only\s+\d+(?:\.\d+)?\s?%\s+of
```

Model-flag sensitivity. 26 of the 69 final papers had received a 'recommended' flag on at least one screening card. Grades of the entries of the other papers (43 entries): A 26, B 8, C 9 (A 60%, C 21%), against A 46, B 13, C 12 (A 65%, C 17%) for all 71 entries. Corpus-level results (Section 3) do not depend on selection.

Claim-matched grade. Seven grade-A entries measure a cost of another kind than the one the paper claims to reduce and are marked A† in Tables 3–7: DeepSeek-VL2 and LLaVA-MoD (two branches) report training time for inference-efficient models; QA-IG reports quantization time; Mirage and Eve report training time; VLA-GSE, a parameter-efficient fine-tuning method, reports inference latency. The other 39 grade-A entries measure a cost of the claimed kind.

Table S8. Prefill memory against batch size (LLaVA-1.5-7B FP16, NVIDIA GeForce RTX 5080, PyTorch 2.11.0). The 576 visual tokens of one image, or an evenly spaced subset of 64 (the memory footprint of any 64-token selection), with 68 text tokens, repeated over the batch; one forward pass with the KV cache. Peak = PyTorch peak allocation, including the 13.2 GB of weights; above weights = peak minus allocation before the pass. Saving = above weights with 576 tokens minus above weights with 64 tokens, from the values shown. Memory in GB of 2^30 bytes; the card has 15.9 GB. The 576-token runs at batch size 8 and 16 exceed it and completed only because the driver spilled into system memory under WSL2, so their run times are not reported.

Batch	Peak, 576 tokens	Peak, 64 tokens	Above weights, 576	Above weights, 64	KV cache, 576	KV cache, 64	Saving
1	13.67	13.33	0.44	0.09	0.31	0.06	0.35
2	14.11	13.41	0.87	0.18	0.63	0.13	0.69
4	14.99	13.59	1.74	0.36	1.26	0.26	1.38
8	16.75	13.95	3.48	0.71	2.52	0.52	2.77
16	20.27	14.68	6.96	1.43	5.03	1.03	5.53

Table S9. Accuracy difference of each method from the unmodified model on the RTX 5080 (method minus unmodified; GQA accuracy, POPE F1 and TextVQA accuracy in points, MME perception in points of 2,000), with paired bootstrap 95% confidence intervals: questions resampled 2,000 times (TextVQA 1,000), and for MME, whose acc+ score is defined per image, images resampled within each category 400 times; TextVQA is scored with the evaluator of the LLaVA code on the questions and OCR reference tokens of LLaVA's evaluation file. The same predictions as Table 8; no new runs. † = the interval excludes zero but the difference does not survive a Holm correction over the 40 differences (normal approximation with the bootstrap standard error). Intervals that include zero: SparseVLM at 192 tokens, POPE; SparseVLM at 192 tokens, TextVQA; DART at 128 tokens, MME; FastV (layer 3) at 288 tokens, MME.

Method, tokens	GQA	POPE F1	TextVQA	MME-P
VisPruner, 128	-3.9 [-4.5, -3.4]	-1.4 [-1.9, -0.8]	-1.2 [-1.9, -0.5]	-52 [-97, -3] †
VisPruner, 64	-5.8 [-6.4, -5.1]	-5.8 [-6.5, -5.0]	-2.7 [-3.5, -2.0]	-140 [-193, -83]
CDPruner (fix), 128	-2.1 [-2.7, -1.6]	+1.5 [+1.1, +2.0]	-2.2 [-3.0, -1.4]	-56 [-107, -4] †
CDPruner (fix), 64	-3.3 [-3.9, -2.7]	+1.6 [+1.1, +2.1]	-3.2 [-4.0, -2.3]	-119 [-173, -69]
SparseVLM, 192	-2.5 [-3.0, -2.0]	-0.3 [-0.8, +0.2]	-0.6 [-1.2, +0.0]	-61 [-110, -13] †

SparseVLM, 64	-8.2 [-8.9, -7.5]	-8.0 [-8.8, -7.1]	-5.2 [-6.0, -4.3]	-220 [-275, -162]
DART, 128	-3.1 [-3.7, -2.5]	-5.3 [-6.0, -4.7]	-1.8 [-2.4, -1.1]	-46 [-90, +1]
DART, 64	-5.5 [-6.2, -4.9]	-11.5 [-12.4, -10.6]	-3.8 [-4.6, -3.0]	-96 [-150, -36]
FastV (layer 3), 288	-1.6 [-2.1, -1.2]	-2.4 [-3.0, -1.9]	-0.6 [-1.1, -0.2] †	-9 [-45, +28]
FastV (layer 3), 64	-8.3 [-9.0, -7.5]	-17.7 [-18.8, -16.5]	-4.4 [-5.3, -3.5]	-208 [-275, -153]

Table S10. Methods against selection without a score at the same token budget, and the two baselines and FastV layer 2 against the unmodified model (RTX 5080; difference with paired bootstrap 95% confidence interval, resampled as in Table S9; * = interval excludes zero; † = not significant after a Holm correction over the 44 comparisons with evenly spaced tokens, computed as in Table S9; the other rows are not corrected). Evenly spaced tokens keep 128 or 64 evenly spaced of the 576 projector outputs; average pooling averages 2×2 or 3×3 blocks of the 24×24 output grid (144 or 64 tokens); FastV layer 2 is the paper's main setting. These settings were run in a second session; in that session the unmodified model took 107.5 ms (Table 8: 106.0 ms) and the ten compressed settings of Table 8 were within -4.4 to +0.9% of their Table 8 TTFT. Raw files: bench/results_pc1/uniform, pool, fastv/acc_k2*, and bench/results_pc1_remeasure.

Comparison	GQA	POPE F1	TextVQA	MME-P
Against evenly spaced tokens, 128				
VisPruner, 128	-0.5 [-1.1, +0.1]	+1.7 [+1.0, +2.5] *	+4.9 [+4.0, +5.9] *	+97 [+31, +153] *
CDPruner (fix), 128	+1.3 [+0.7, +1.9] *	+4.6 [+3.9, +5.4] *	+4.0 [+3.2, +4.9] *	+92 [+42, +147] *
DART, 128	+0.3 [-0.3, +0.9]	-2.2 [-3.1, -1.4] *	+4.4 [+3.5, +5.3] *	+103 [+43, +155] *
Average pooling, 144	-1.0 [-1.5, -0.4] *	-0.6 [-1.3, +0.1]	-0.8 [-1.6, +0.0]	-18 [-72, +40]
Against evenly spaced tokens, 64				
VisPruner, 64	+0.6 [-0.1, +1.3]	+3.1 [+2.1, +4.0] *	+6.7 [+5.7, +7.8] *	+49 [-23, +114]
CDPruner (fix), 64	+3.0 [+2.4, +3.7] *	+10.5 [+9.6, +11.4] *	+6.2 [+5.2, +7.2] *	+69 [+4, +138] * †
SparseVLM, 64	-1.9 [-2.6, -1.1] *	+0.9 [-0.1, +1.9]	+4.2 [+3.2, +5.2] *	-31 [-115, +38]
DART, 64	+0.8 [+0.1, +1.5] * †	-2.6 [-3.7, -1.6] *	+5.6 [+4.5, +6.6] *	+92 [+21, +168] * †
FastV (layer 3), 64	-1.9 [-2.7, -1.1] *	-8.8 [-9.9, -7.7] *	+5.0 [+4.0, +6.0] *	-20 [-97, +50]
FastV (layer 2), 64	-4.0 [-4.8, -3.1] *	-17.2 [-18.6, -15.8] *	+6.4 [+5.4, +7.5] *	-61 [-147, +11]
Average pooling, 64	-1.3 [-2.0, -0.6] *	-0.4 [-1.3, +0.6]	-1.9 [-2.8, -1.1] *	-174 [-244, -111] *
FastV layer 2 against layer 3, same budget				
FastV (layer 2), 288	-0.2 [-0.6, +0.1]	-0.9 [-1.3, -0.5] *	+0.5 [+0.1, +0.9] *	-15 [-34, +0]
FastV (layer 2), 64	-2.1 [-2.6, -1.6] *	-8.4 [-9.4, -7.4] *	+1.4 [+0.8, +2.1] *	-41 [-83, +4]
Against the unmodified model				
Evenly spaced, 128	-3.4 [-4.0, -2.8] *	-3.1 [-3.8, -2.4] *	-6.2 [-7.1, -5.2] *	-148 [-210, -95] *
Evenly spaced, 64	-6.3 [-7.0, -5.6] *	-8.9 [-9.8, -8.0] *	-9.4 [-10.5, -8.2] *	-188 [-257, -117] *
Average pooling, 144	-4.4 [-5.0, -3.8] *	-3.7 [-4.4, -3.0] *	-7.0 [-8.0, -6.0] *	-167 [-222, -110] *
Average pooling, 64	-7.6 [-8.4, -6.9] *	-9.2 [-10.2, -8.3] *	-11.3 [-12.4, -10.2] *	-362 [-441, -277] *
FastV (layer 2), 288	-1.9 [-2.3, -1.4] *	-3.3 [-3.9, -2.7] *	-0.1 [-0.5, +0.3]	-24 [-62, +12]
FastV (layer 2), 64	-10.3 [-11.1, -9.5] *	-26.1 [-27.4, -24.8] *	-3.0 [-3.7, -2.2] *	-249 [-312, -190] *

Table S11. torch.compile on the unmodified LLaVA-1.5-7B (576 visual tokens) on the RTX 5080, measured in one session with the protocol of Table 8 (500 GQA questions × 5 runs; TTFT = median of the five run medians; IQR and range over the five run medians). Default inductor backend without CUDA graphs, which would have to be re-recorded for every prompt length; the language model is compiled with dynamic shapes, and the per-layer integer layer index is treated as dynamic so that the 32 layers share one compiled graph (no recompilation during the measured runs). Compilation and warm-up (12 prompts of different lengths) are excluded from the timings. The compression methods were not compiled: their selection code would have to be changed, so it would no longer be the code as released. Code: bench/methods/base_compile.py; raw files and logs: bench/results_pc1_compile.

Setting	TTFT (ms)	IQR (ms)	Range of run medians (ms)	End-to-end (ms)	Energy (J/sample)	Peak memory (GB)	Compile + warm-up
Eager (the stack of Table 8)	110.1	2.1	108.4–110.7	145.9	44.1	13.95	—
Vision encoder compiled	113.6	10.1	107.4–121.8	149.3	44.0	13.95	13 s
Vision encoder and language model compiled	102.6	2.5	102.2–105.7	138.2	41.9	13.95	93 s

Monthly series. Across all VLM papers the strict execution-time share was 0.4% in January–August 2024, 2.1% in September–December 2024, 2.0% in 2025 and 2.9% in January–August 2026 (Fig. S1a); the loose share followed the same course (0.9, 3.8, 3.3 and 4.6%; Fig. S1b). Fisher's exact tests between adjacent periods, whose boundaries were chosen after inspecting the series and which are not corrected for multiplicity, give ×5.24 (p = 4.3e-05), ×0.99 (p = 0.91) and ×1.43 (p =

0.0016). Within the efficiency pool the strict share rose from 3.2% to 18.1% between the two parts of 2024 and then stayed between 16.1% and 17.5% (2025 against 2026, ×1.09, p = 0.47), so the rise in 2026 across all VLM papers comes mainly from the growing share of efficiency papers (Section 3.4). Fig. S2 shows the strict share of every branch by quarter. Script: monthly_trend.py; counts: main_survey/monthly_trend.

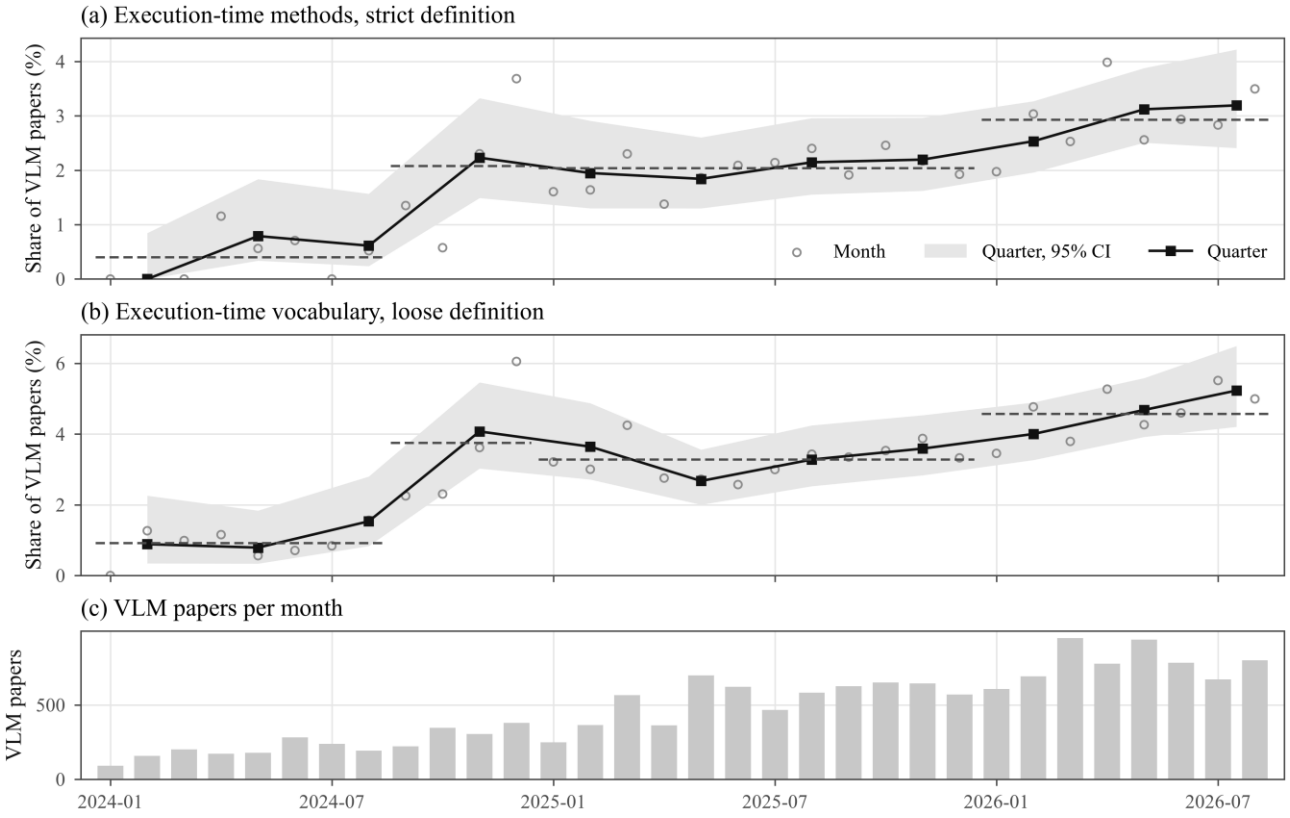


Fig. S1. Execution-time branch (4.4) by month across all 15,394 VLM papers. (a) Strict and (b) loose share per month (open circles) and per quarter (squares, with Wilson 95% confidence band), and the share over each period compared in Section 3.4 (dashed: January–August 2024, September–December 2024, 2025, January–August 2026). (c) VLM papers per month. Rules as in Table 2; the yearly counts equal those of Table 2. Panel (a) is shown as Fig. 2c of the main text.

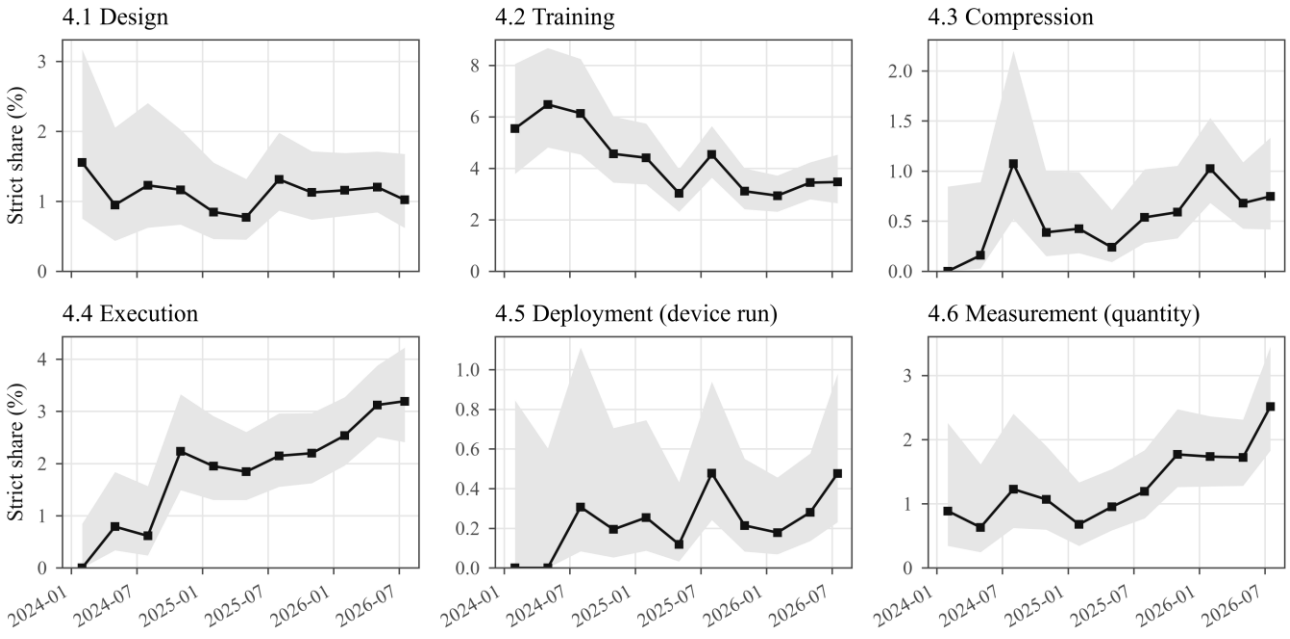


Fig. S2. Strict share of each branch by quarter across all VLM papers, with Wilson 95% confidence bands; strict definitions as in Table 2 (a proposed method for 4.1–4.4, a run on a named device for 4.5, a quantity with a unit for 4.6).

Table S12. Strict execution-time (4.4) and training-time (4.2) shares under the specifications of Section 3.4 (Fig. 2d of the main text). All VLM papers unless the efficiency pool is named; k/n = matching papers / papers in the earlier → the later period (2024 → January–August 2026 unless stated); 95% CI by the Katz log method, except the corrected row (bootstrap; Section 3.4); p = two-sided Fisher's exact test, not corrected across rows. Broader vocabulary: Table S7. Script: spec_growth.py; values: main_survey/spec_growth.

Specification	4.4 k/n	4.4 ×growth (95% CI)	4.4 p	4.2 k/n	4.2 ×growth (95% CI)	4.2 p
Strict rule, 2026 vs 2024 (Table 2)	32/2,767 → 182/6,220	×2.53 (1.74–3.67)	1.1e-07	153/2,767 → 203/6,220	×0.59 (0.48–0.72)	7.6e-07
Broader execution vocabulary	49/2,767 → 195/6,220	×1.77 (1.30–2.41)	1.8e-04	—	—	—
Verbs propose, present, introduce	30/2,767 → 181/6,220	×2.68 (1.83–3.94)	2.4e-08	147/2,767 → 192/6,220	×0.58 (0.47–0.72)	7.5e-07
Subject “we” only	30/2,767 → 181/6,220	×2.68 (1.83–3.94)	2.4e-08	143/2,767 → 189/6,220	×0.59 (0.48–0.73)	1.5e-06
Without title matches	21/2,767 → 122/6,220	×2.58 (1.63–4.10)	9.7e-06	114/2,767 → 164/6,220	×0.64 (0.51–0.81)	2.7e-04
Same months (Jan–Aug)	6/1,514 → 182/6,220	×7.38 (3.28–16.62)	2.2e-11	94/1,514 → 203/6,220	×0.53 (0.41–0.67)	4.6e-07
2026 restricted to Jan–Jun	32/2,767 → 135/4,748	×2.46 (1.68–3.60)	8.4e-07	153/2,767 → 152/4,748	×0.58 (0.46–0.72)	1.5e-06
2026 vs 2025	131/6,407 → 182/6,220	×1.43 (1.15–1.79)	0.0016	237/6,407 → 203/6,220	×0.88 (0.73–1.06)	0.19
Within the efficiency pool	30/268 → 165/941	×1.57 (1.09–2.26)	0.014	44/268 → 74/941	×0.48 (0.34–0.68)	9.5e-05
Corrected for precision and recall	—	×1.88 (0.79–4.38)	—	—	—	—